

Amazon Reviews Classifier

Caleb Walton





What is the Problem?

- There is an uncountable amount of online product reviews
- Computer programs are capable of dealing with huge amounts of data
- Is it possible determine the sentiment of every topic in the review (and not simply whether the review is positive)?



My Approach To This Problem

- I created a program that tries to address this problem
- This program does not use any pre-existing models; rather, it tries to develop a classification system from scratch
- Topics aren't simply classified as positive or negative, but a positive or negative number symbolizes their sentiment



My Data Source

- McAuley-Lab's 2023 Amazon Reviews Dataset
- Excerpts were used to create my training and testing data in the form of CSV files
- Link: <https://huggingface.co/datasets/McAuley-Lab/Amazon-Reviews-2023>

This is a large-scale **Amazon Reviews** dataset, collected in **2023** by [McAuley Lab](#), and it includes rich features such as:

1. **User Reviews** (*ratings, text, helpfulness votes, etc.*);
2. **Item Metadata** (*descriptions, price, raw image, etc.*);
3. **Links** (*user-item / bought together graphs*).

Excerpt Creation Code



```
# RUN THIS FIRST
!pip install datasets==3.6.0
```

```
import datasets
from datasets import load_dataset
import csv

# creating training data but using csv
# example: section="raw_review_All_Beauty", start_num_lines=10000, end_num_lines=20000, file_name="test_write.csv"
# training data will start at review #10001 for each category
def create_csv_data(section, start_num_lines, end_num_lines, file_name):

    open_file = open(file_name, "w")
    open_file.close()

    dataset = load_dataset("McAuley-Lab/Amazon-Reviews-2023", section, split=f"full[{start_num_lines}:{end_num_lines}]", trust_remote_code=True)

    data_list = [[dataset[i]["rating"], dataset[i]["title"], dataset[i]["text"]] for i in range(end_num_lines - start_num_lines)]

    csv_fields = ["Rating", "Title", "Text"]

    with open(file_name, 'a') as f:
        csv_writer = csv.writer(f)
        csv_writer.writerow(csv_fields)
        csv_writer.writerows(data_list)

    read_fields = []
    read_rows = []

    with open(file_name, 'r') as f:
        csv_reader = csv.reader(f)

        read_fields = next(csv_reader)
        for row in csv_reader:
            read_rows.append(row)

    print(read_fields)
    for review in read_rows:
        print(review)
```

```
create_csv_data("raw_review_Arts_Crafts_and_Sewing", 10000, 35000, "acs_csv_train.csv")
create_csv_data("raw_review_Arts_Crafts_and_Sewing", 40000, 50000, "acs_csv_test.csv")
```

Show hidden output

```
create_csv_data("raw_review_Books", 10000, 35000, "books_csv_train.csv")
create_csv_data("raw_review_Books", 40000, 50000, "books_csv_test.csv")
```

Show hidden output

```
create_csv_data("raw_review_Cell_Phones_and_Accessories", 10000, 35000, "cpa_csv_train.csv")
create_csv_data("raw_review_Cell_Phones_and_Accessories", 40000, 50000, "cpa_csv_test.csv")
```

Show hidden output



Training the Classifier

- For every CSV file in the train_data folder, do the following:
 - Determine the weight that the words will have on the overall sum of words based on the review score
 - Every time the program comes across a word (excluding stop words), add the weight to a running sum for the corresponding word
- Then, for every word analyzed by the algorithm, divide the total sum by the number of times the word was encountered. This is the word's sentiment value
- These values are stored in a dictionary. Filter the dictionary so there are no words that do not occur often enough


```
# Train Phase
# This code cell uses multiple CSV files to create word sentiment weights.
# This code is to make it so this program learns the sentiment of all words (except stop words).
```

```
# import and download the required libraries
```

```
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
import csv
import os
from nltk.corpus import wordnet as wn
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

```
nltk.download("wordnet")
nltk.download("stopwords")
nltk.download("punkt")
nltk.download("punkt_tab")
```

```
# create list of stop words
stop_words_list = list(stopwords.words("english"))
stop_words_list = stop_words_list + ["a", "an", "and", "are", "as", "at", "be", "but", "by", "can", "could", "did", "do", "does", "for", "from", "had", "has", "have", "he", "her", "his", "hundred", "if", "in", "into", "is", "it", "its", "me", "more", "most", "my", "no", "nor", "not", "of", "on", "or", "over", "she", "so", "some", "that", "the", "there", "they", "this", "to", "too", "us", "was", "we", "were", "with", "without", "you", "your", "yourself"]
```

```
train_data = os.listdir("train_data")
```

```
print(train_data)
```

```
sentiment_dict = {}
```

```
# start analyzing the review data
```

```
for name in train_data:
    dir = "train_data/" + name
    print(f"Now analyzing {dir}")
```

```
csv_file = open(dir, 'r')
csv_reader = csv.reader(csv_file)
```

```
read_fields = next(csv_
read_rows = []
for row in csv_reader:
    read_rows.append(row)
```

```
# print(read_rows)
```

```
csv_file.close()
```

```
# start analyzing a review
```

```
for review in read_rows:
    w_weight = 0
    if review[0] == "1.0":
        w_weight = -35
    elif review[0] == "2.0":
        w_weight = -20
    elif review[0] == "3.0":
```

```
for review in read_rows:
    w_weight = 0
    if review[0] == "1.0":
        w_weight = 35
    elif review[0] == "2.0":
        w_weight = 20
    elif review[0] == "3.0":
        w_weight = 10
    elif review[0] == "4.0":
        w_weight = 3
    else:
        w_weight = 10
```

```
# tokenize the review's title and text
rev_title = word_tokenize(review[1].lower())
rev_text = word_tokenize(review[2].lower())
```

* in the review title and text, add to a cumulative sum of weights for each eligible word

```
for word in rev_title:
    if word not in stop_words_list:
        if word not in sentiment_dict:
            sentiment_dict[word] = [w_weight, 1]
        else:
            sentiment_dict[word][0] += w_weight
            sentiment_dict[word][1] += 1
```

```
for word in rev_text:
    if word not in stop_words_list:
        if word not in sentiment_dict:
            sentiment_dict[word] = [w_weight, 1]
        else:
            sentiment_dict[word][0] += w_weight
            sentiment_dict[word][1] += 1
```

```
# append the average into the sentiment dictionary
for node in sentiment_dict:
    sentiment_dict[node].append((sentiment_dict[node][0] / sentiment_dict[node][1])

# filter the dictionary for more useful words
f = sentiment_dict.items()
for node, sentiment in f:
    if sentiment[1] > 100
```

```
# display the results
print(sorted(f_sentiment_dict.items(), key = lambda d: d[1][2]))
```

```
# put the model's values into a CSV file
data_list = [(node, f_sentiment_dict[node][0], f_sentiment_dict[node][1], f_sentiment_dict[node][2]) for node in f_sentiment_dict]

csv_fields = ["word", "Sum", "Occurrences", "Average"]
```

```

# write the entire sentiment dictionary into CSV file. The resulting file represents the drastic difference between the sizes of the training data and the model's weights.
with open("write_dict_into_csv.csv", "w") as f:
    csv_writer = csv.writer(f)
    csv_writer.writerow(csv_fields)
    csv_writer.writerows(data_list)

```

```
# test phase will need this function, run this cell too
```

```
def is_noun_or_unknown(word):
    # words unknown to wordnet automatically pass
    if wn.synsets(word) == []:
        return True
    else:
        # words with a high enough percentage of noun synsets to all of its synsets (>50%) pass
        noun_num = 0
        other_num = 0
        for syn in wn.synsets(word):
            if syn.name() in str('n'):
                noun_num += 1
            else:
                other_num += 1
        if noun_num / (noun_num + other_num) > 0.59:
            return True
        else:
            return False
```

'junk', -25469, 1890, -23.366055048581756], 'waste', -64544, 2766, -23.334779464941313], 'worthless', -6541, 286, -22.87062937062937], 'refund', -13025, 589, -22.113751222410087], 'garbage', -18732, 856, -22.88317575009346], 'creativity', [2819, 284, 9.92605633808281], 'busy', [1662, 1938, 200, 9.69], 'metrics', [1916, 207, 9.25603864732295], 'junkies', [608, 787, 9.2088486562942], 'art', [1603, 176, 9.16]



Testing the Classifier

- The dictionary created previously will from now on be called “sentiment_dict.”
- The dictionary we will store the average sentiment of topics in will be called “like_dislike_dict.”
- For every CSV file in the test_data folder:
 - Analyze the surrounding words around every word that is determined to be a noun or an unknown word. Every word’s sentiment value from sentiment_dict is added to the target word’s sentiment sum in like_dislike_dict.
 - Words that are closer to the target word will have more impact on the sum, while words that are further away will have less impact
 - If the target word is not in the review title, the words in the review title are not discounted by being further away from the target word and therefore have the most impact
- Then, for every word in like_dislike_dict, divide the total sum by the number of times the sum was modified. This is the average sentiment of the word (or topic).

Testing Classifier Code

```
# Test Phase
# This testing code uses sentiment_dict to determine the sentiment of nouns in a set of reviews
# It is recommended to test sets of reviews of single categories for better analysis of specific product markets

test_data = os.listdir("test_data")

print(test_data)

all_id_dicts_n = []
all_id_dicts_p = []
like_dislike_dict = {}

# Go to every file in the test_data folder
for name in test_data:
    like_dislike_dict = {}
    dir = "test_data/" + name
    print(f"Now analyzing {dir}")

    try:
        csv_file = open(dir, "r")
    except IOError:
        continue

    csv_reader = csv.reader(csv_file)

    read_fields = next(csv_reader)
    read_rows = []
    for row in csv_reader:
        read_row.append(row)

    # print(read_rows)

    csv_file.close()

    # minimum word occurrence for sect[i] to consider and consider word threshold for sect[i] to consider
    min_word_occurrence = 140
    consider_word_threshold = sorted(f_sentiment_dict.items(), key=lambda d: -d[1])[0][1] + 1
    # print(consider_word_threshold)

    rev_count = 0
    # start reading through the reviews in a file
    for rev in read_rows:
        # create tokenized instances of review title and text
        rev_tokens = [word_tokenize(rev[1].lower()), word_tokenize(rev[2].lower())]
        sect_num = 0
        for sect in rev_tokens:
            # evaluate the sentiment of the target word
            for i in range(len(sect)):
                # evaluate for noun or unknown word
                if is_noun_or_unknown(word):
                    if sect[i] not in stop_words_list:
                        if (sect[i] not in f_sentiment_dict) or (f_sentiment_dict[sect[i]] < consider_word_threshold):
                            # use the sentiment of surrounding words to weigh the target word
                            for j in range(len(sect)):
                                if (sect[j] not in f_sentiment_dict) or (f_sentiment_dict[sect[j]] < consider_word_threshold):
                                    # use the sentiment of surrounding words to weigh the target word
                                    for j in range(len(sect)):
                                        try:
                                            if (sect[j] not in stop_words_list) and (sect[j] in f_sentiment_dict):
                                                if (sect[i] not in like_dislike_dict):
                                                    if f_sentiment_dict[sect[j]] > min_word_occurrence:
                                                        like_dislike_dict[sect[i]] = (f_sentiment_dict[sect[j]] * 2 ** (abs(i - j)), 1)
                                                    else:
                                                        if f_sentiment_dict[sect[j]] > min_word_occurrence:
                                                            # print(sect[i], like_dislike_dict[sect[i]])
                                                            like_dislike_dict[sect[i]] += f_sentiment_dict[sect[j]] / (2 ** (abs(i - j)))
                                                            like_dislike_dict[sect[i]][1] += 1
                                                            # print(sect[i], like_dislike_dict[sect[i]])
                                except OverflowError:
                                    # print("Overflow encountered while processing " + str(sect[j]) + " for " + str(sect[i]))
                                    break
                            if sect_num == 1:
                                for j in range(len(rev_tokens[0])):
                                    try:
                                        if (rev_tokens[0][j] not in stop_words_list) and (rev_tokens[0][j] in f_sentiment_dict):
                                            if (sect[i] not in like_dislike_dict):
                                                if f_sentiment_dict[rev_tokens[0][j]] > min_word_occurrence:
                                                    like_dislike_dict[sect[i]] = (f_sentiment_dict[rev_tokens[0][j]] * 2, 1)
                                                else:
                                                    if f_sentiment_dict[rev_tokens[0][j]] > min_word_occurrence:
                                                        # print(sect[i], like_dislike_dict[sect[i]])
                                                        like_dislike_dict[sect[i]] += f_sentiment_dict[rev_tokens[0][j]] * 2
                                                        like_dislike_dict[sect[i]][1] += 1
                                                        # print(sect[i], like_dislike_dict[sect[i]])
                                except OverflowError:
                                    # print("Overflow encountered while processing " + str(rev_tokens[0][j]) + " for " + str(sect[i]))
                                    break
                            if sect_num == 1:
                                # print(rev_count)
                                rev_count += 1
                                # append the averages of words into the dictionary
                                for node in like_dislike_dict:
                                    like_dislike_dict[node].append(like_dislike_dict[node][0] / like_dislike_dict[node][1])
                                # filter the dictionary so there are less irrelevant words
                                f_like_dislike_dict = {node: like_dislike_dict[node] for node in like_dislike_dict if like_dislike_dict[node][1] > 350}

                                # display the results
                                # print(like_dislike_dict.items())
                                # The same topic sentiment dictionary is printed twice, but one is sorted from most negative to most positive
                                # and the other is vice versa
                                print(sorted(f_like_dislike_dict.items(), key = lambda d: d[1][2]),)
                                print(sorted(f_like_dislike_dict.items(), key = lambda d: -d[1][2]))

                                all_id_dicts_n.append(sorted(f_like_dislike_dict.items(), key = lambda d: d[1][2]))
                                all_id_dicts_p.append(sorted(f_like_dislike_dict.items(), key = lambda d: -d[1][2]))

... ["plg_csv_test.csv", "books_csv_test.csv", "ggf_csv_test.csv", "matv_csv_test.csv", "hk_csv_test.csv", "cpa_csv_test.csv"]
Now analyzing test_data/plg_csv_test.csv
[('junk', [-2620.4977784065427, 2352, -1.1141572153089043]), ('crap', [-767.9551505599518, 772, -0.9947605577201448]), ('buyer', [-279.8406452549027, 627, -0.4463166543140394]), ('nope', [-220.8916472680355, 537, -0.4080640091748297]), ('garbage', [-792.283772244]), ('loves', [2261.61156361414, 1881, 1.202345328875141]), ('five', [5707.601807985672, 5847, 0.9761590230863113]), ('sister', [342.7726605510091, 354, 0.9682843518390087]), ('art', [279.853968659967, 373, 0.7502787363538063]), ('holiday', [353.7662732951904, 494])
Now analyzing test_data/books_csv_test.csv
[('trash', [-189.0992700183926, 416, -0.4546555290959194]), ('meh', [-167.58236608792593, 576, -0.290941608790889916]), ('garbage', [-229.33685072473145, 888, -0.25826221928460746]), ('disappointment', [-274.7572262119953, 1472, -0.1866557432880118]), ('nope', [-1652.619212507322, 711, 2.324358948674152]), ('five', [17054.768377515058, 10215, 1.669580049487524]), ('grandson', [2123.3480511844223, 1475, 1.4395580080029982]), ('granddaughter', [1839.7756700200391, 1357, 1.3557668902137354]), ('grandkids', [459.1924517496131, 469, -0.9790883832614352]), ('buyer', [-304.3189991446401, 445, -0.6838629194261575]), ('refund', [-756.3329575842472, 1156, -0.6542672643462346]), ('nope', [-317.25755809523827, 543, -0.584268062790494]), ('meh', [-405.49017908]), ('son', [1212.0013769684144, 1708, 0.7100710637988374]), ('birthday', [716.1603457571426, 1085, 0.660056152093481]), ('love', [32604.04728317738, 53586, 0.608443386018142]), ('wife', [1725.9590380264226, 2863, 0.6828498211758374]), ('thanks', [1262.5380943562])
Now analyzing test_data/matv_csv_test.csv
[('garbage', [-370.15960834332476, 413, -0.8962702260128929]), ('meh', [-252.89623029735807, 530, -0.47716269067426053]), ('crap', [-438.514717178284527, 1067, -0.401979116990115]), ('trash', [-360.699951091144, 969, -0.37223937161105713]), ('buyer', [-122.063526]), ('five', [16207.83277838756, 8428, 1.9238935902211386]), ('delivery', [1103.499636354103, 1025, 1.0765850110764978]), ('binge', [1357.1228402862596, 1398, 0.9707602577154932]), ('grandson', [911.219363664554, 939, 0.9704146500175244]), ('grandchildren', [722])
Now analyzing test_data/hk_csv_test.csv
[('junk', [-2036.7283375919167, 4424, -0.4603816314629106]), ('buyer', [-563.1755075798413, 1383, -0.407212483385259]), ('refund', [-516.062509074161, 1475, -0.34987298366604486]), ('crap', [-441.544458184401, 1460, -0.3027016813824932]), ('garbage', [-963.44]), ('delivery', [-512.5164567847545, 545, 0.9403971684123936]), ('loves', [2888.5970083130946, 3912, 0.7383939312804434]), ('duvet', [634.592251148711, 991, 0.6403554582006772]), ('medicine', [322.6273962538194, 510, 0.6326027377525784]), ('order', [533.5394527])
Now analyzing test_data/cpa_csv_test.csv
[('junk', [-1845.7323754720903, 1382, -1.3355516458065778]), ('trash', [-883.3952402761002, 1116, -0.7915727959463336]), ('garbage', [-1416.0132729578172, 1980, -0.7151582186655643]), ('refund', [-1025.6301729797076, 2044, -0.5017760614779392]), ('crap', [-445.6]), ('five', [6045.239636887342, 3320, 1.8280553123154645]), ('loves', [3170.9902793946150, 2335, 1.3580258155865592]), ('gift', [2306.5826283911424, 2928, 0.7877672911171790]), ('delivery', [1274.636366369556, 1706, 0.7471491206241944]), ('gifts', [301.314600126])
```



Ways to Analyze the Data

- The program is able to do various forms of analysis of like_dislike_dict after the testing phase
 - Can create a pandas dataframe listing the top 10 words with the most (or least) sentiment
 - Search for a specific word in a specific category
 - Search for a word in every category and create a bar chart comparing the sentiment values for each category in which a sentiment value for the word is found

Data Analysis Code

```
# create a pandas table after finding what reviewers like and dislike
# the pandas table is sorted from highest to lowest sentiment, so it's basically a top 10 (mostly) topics
```

```
dict_to_df_p = [(all_id_dicts_p[i][j][0], all_id_dicts_p[i][j][1]) for i in range(len(all_id_dicts_p))] for i in range(len(all_id_dicts_p))
dict_to_df_n = [(all_id_dicts_n[i][j][0], all_id_dicts_n[i][j][1]) for i in range(len(all_id_dicts_n))] for i in range(len(all_id_dicts_n))
```

```
test_data_pfixs = [name.split(",")[0] for name in test_data if ".lysm" not in name]
```

```
id_df_p = pd.DataFrame(dict_to_df_p, index = test_data_pfixs, columns = np.arange(1, max([len(all_id_dicts_p[i]) for i in range(len(all_id_dicts_p))]) + 1))
id_df_n = pd.DataFrame(dict_to_df_n, index = test_data_pfixs, columns = np.arange(1, max([len(all_id_dicts_n[i]) for i in range(len(all_id_dicts_n))]) + 1))
id_df_p
```

	1	2	3	4	5	6	7	8	9	10 ...
df	(son,	(birthday,	(love,	(wife,	(thanks,	(loves,	(christmas,	(facility,	(antioxidants,	(favorite,
hk	0.710071063790374),	0.6600555182093481),	0.6004433060103742),	0.6028490211156374),	0.602355061847792),	0.5931375422752450),	0.5816554706480345),	0.5762835651702457),	0.5715325600151525),	0.5680516271917869),
matv	0.9403971684123936),	0.7363839182084434),	0.6403554502006772),	0.6326027373257514),	0.6125596473009892),	0.5975966097524874),	0.57835746507350),	0.5762205178734144),	0.5690018078789126),	0.5617908214676171),
cpa	1.9230935902211386),	1.0765850110764978),	0.9707602577154932),	0.9704146500175244),	0.889294824345108),	0.8448105399114477),	0.832820274491077),	0.6817470310031887),	0.6594987545466543),	0.6147232383055863),
plg	1.820853123154645),	1.3580258155695592),	0.78767291171795),	0.7471491022841944),	0.7349136588442776),	0.6634097283598712),	0.661882415720453),	0.652892440494391),	0.630979001219136),	0.627919706139134),
books	1.202345328751411),	0.9761590233603513),	0.8628435183900057),	0.7502787363300653),	0.709751546745491),	0.7086772797102065),	0.6916160546322199),	0.6914377558183465),	0.6775862707652873),	

```
# After running the previous cell, create the same table but in reverse order.
id_df_n
```

	1	2	3	4	5	6	7	8	9	10 ...
df	(garbage,	(buyer,	(refund,	(nope,	(meh,	(junk,	(ok,	(crap,	(trash,	(disappointment,
hk	-0.979083832614352),	-0.6838628194261575),	-0.654267264362246),	-0.584268062700494),	-0.551687184402805),	-0.3951656389525453),	-0.3508717441984415),	-0.30265664991333074),	-0.25074697268915089),	-0.14786984335262929),
matv	-0.4603616314620186),	-0.4072125483565259),	-0.3498729836694488),	-0.30279168132614952),	-0.22643146821001192),	-0.20956145595952340),	-0.18715131196961455),	-0.1816611330071091),	-0.09991540435777049),	-0.0965573505760605),
cpa	-0.8862702269128925),	-0.47716289867426053),	-0.410879116980115),	-0.3722937161100513),	-0.32637306402200374),	-0.2176110450413264),	-0.21655724647891653),	-0.14477299430056314),	-0.133897531905421),	-0.10884500665472338),
plg	-1.3355516465065778),	-0.7915727959463336),	-0.715150216665643),	-0.5817760141779392),	-0.49431680755097956),	-0.39424962145439563),	-0.23704507405012223),	-0.2315532216725238),	-0.20572294189757087),	-0.1753201448764309),
books	-1.1415721308304443),	-0.8947605772914448),	-0.4463166543140394),	-0.4095489114629177),	-0.33585200862298417),	-0.3290831060381818),	-0.318743027232959),	-0.2547795309741163),	-0.20648319603360303),	-0.18786981415312072),

```
# search for a specific word's score
# remember to leave no spaces in or around the word
```

```
a_category = str(input("Enter the category:")).lower()
a_word = str(input("Enter the word:")).lower()
```

```
cat_index = 0
for i in range(len(test_data_pfixs)):
    if a_category == test_data_pfixs[i]:
        cat_index = i
        break

for j in range(len(dict_to_df_p[cat_index])):
    # print(dict_to_df_p[cat_index][j][0])
    if dict_to_df_p[cat_index][j][0] == a_word:
        print(dict_to_df_p[cat_index][j])
        break
```

```
Enter the category:cpa
Enter the word:garbage
('garbage', -0.715150216665643)
```

```
# search for a specific word's score across every category and create a bar graph
# remember to leave no spaces in or around the word
```

```
a_word = str(input("Enter the word:")).lower()
```

```
all_word_scores = []
```

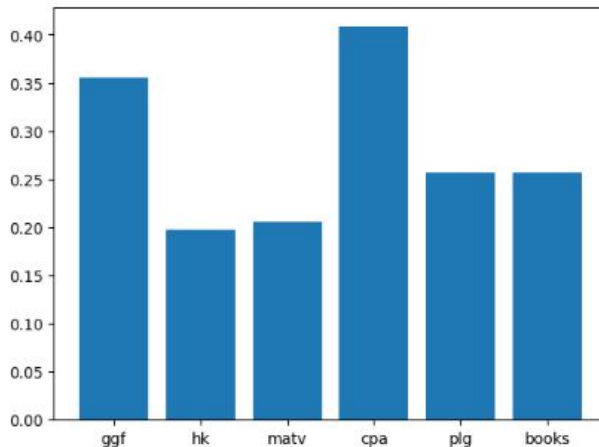
```
for i in range(len(dict_to_df_p)):
    for j in range(len(dict_to_df_p[i])):
        # print(dict_to_df_p[cat_index][j][0])
        if dict_to_df_p[i][j][0] == a_word:
            all_word_scores.append([test_data_pfixs[i], dict_to_df_p[i][j][1])
            break
```

```
print(all_word_scores)
```

```
cats = [node[0] for node in all_word_scores]
scores = [node[1][1] for node in all_word_scores]
```

```
plt.bar(cats, scores)
plt.title(f'Scores of the word \"{a_word}\"')
plt.show()
```

Scores of the word "book"





Analysis Dashboard

- Dashboard is on the next page

Top 10 Positive and Negative Words by Category + Scores of Words by Category

Legend

ggf

Grocery and
Gourmet Food

hk

Home and
Kitchen

matv

Movies and TV

cpa

Cell Phones and
Accessories

plg

Patio Lawn and
Garden

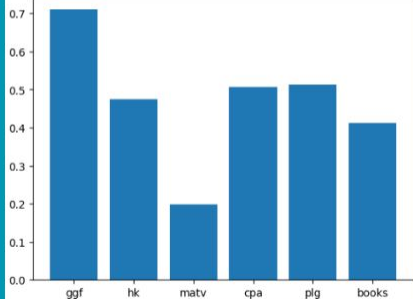
books

Books

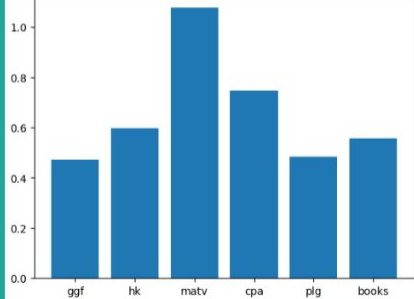
	1	2	3	4	5	6	7	8	9	10
ggf	(garbage, -0.9790883832614352)	(buyer, -0.6838629194261575)	(refund, -0.6542672643462346)	(nope, -0.5842680627904394)	(meh, -0.5516873184402805)	(junk, -0.3951656389525453)	(ok., -0.3506717441984415)	(crap, -0.30265664991333074)	(trash, -0.2507469726915089)	(disappointment, -0.1478698433529289)
hk	(junk, -0.4603816314629106)	(buyer, -0.4072129483585259)	(refund, -0.34987298366604486)	(crap, -0.3027016813824932)	(garbage, -0.23643146821061192)	(steelcase, -0.20056145859585248)	(nope, -0.18715131109663455)	(ugh, -0.18166113300701991)	(seatback, -0.09991548438577649)	(keg, -0.0965573505796805)
matv	(garbage, -0.8962702260128929)	(meh, -0.47716269867426053)	(crap, -0.410979116990115)	(trash, -0.37223937161105713)	(buyer, -0.32637306492200374)	(blah, -0.21761104504132264)	(execution, -0.21655724647891653)	(disappointment, -0.14477299430856314)	(joke, -0.1338997531905421)	(ok., -0.10884580665472338)
cpa	(junk, -1.3355516465065778)	(trash, -0.7915727959463336)	(garbage, -0.7151582186655643)	(refund, -0.5017760141779392)	(crap, -0.49431680755097956)	(disappointment, -0.39424982145439563)	(meh, -0.23704507405012223)	(money, -0.2315532216725238)	(buyers, -0.20572294189757087)	(joke, -0.1753201448784309)
plg	(junk, -1.1141572153089043)	(crap, -0.9947605577201448)	(buyer, -0.4463166543140394)	(nope, -0.409854091748297)	(garbage, -0.33858280865299417)	(refund, -0.3290881068838118)	(trash, -0.3176743622732955)	(money, -0.2547758390741163)	(joke, -0.20648319605369203)	(disappointment, -0.18798991415312072)
books	(trash, -0.4545655290959194)	(meh, -0.29094160780889916)	(garbage, -0.25826221928460746)	(disappointment, -0.18665572432880118)	(nope, -0.14466366061507402)	(brooklyn, -0.12304540681682555)	(junk, -0.11755259800257803)	(battery, -0.11083238233522079)	(europacific, -0.1022768035527149)	(stocks, -0.10130790814992693)

	1	2	3	4	5	6	7	8	9	10
ggf	(son, 0.7100710637988374)	(birthday, 0.6600556182093481)	(love, 0.6084433860183142)	(wife, 0.6028498211758374)	(thanks, 0.6023559610477792)	(loves, 0.5931375422752458)	(christmas, 0.5816554786849345)	(acidity, 0.5782835851702457)	(antioxidants, 0.5715325569315125)	(favorite, 0.5690516271917869)
hk	(delivery., 0.9403971684123936)	(loves, 0.7383939182804434)	(duvet, 0.6403554502006772)	(medicine, 0.6326027377525784)	(order., 0.6125596473009692)	(delivery, 0.5975966097524874)	(birthday, 0.57835746507358)	(turquoise, 0.5762205178734144)	(alot, 0.5698018079789126)	(tablecloth, 0.5617908214676117)
matv	(five, 1.9230935902211386)	(delivery, 1.0765850110764978)	(binge, 0.9707602577154932)	(grandson, 0.9704146500175244)	(grandchildren, 0.889294824345108)	(condition, 0.8448105399114477)	(grandkids, 0.832820274491077)	(stars, 0.6817470310031867)	(dogs, 0.6594987545466543)	(thriller, 0.6147223283055863)
cpa	(five, 1.8208553123154645)	(loves, 1.3580258155865592)	(gift, 0.7877672911171798)	(delivery, 0.7471491022041944)	(gifts, 0.7349136588442776)	(stores, 0.6634097283598712)	(love, 0.661682415720453)	(kitchen, 0.652892449494391)	(stars, 0.630979801219136)	(daughter, 0.627919786139134)
plg	(loves, 1.202345328875141)	(five, 0.976159023086313)	(sister, 0.9682843518390087)	(art, 0.7502787363538063)	(holiday, 0.7219719863867735)	(gift, 0.7097551546743491)	(fun, 0.7086772797102895)	(hubby, 0.6916160546322199)	(fairly, 0.6914377561883405)	(colors, 0.6775862707625873)
books	(recipient, 2.324358948674152)	(five, 1.669580849487524)	(grandson, 1.4395580080029982)	(granddaughter, 1.3557668902137354)	(grandkids, 1.199040399123854)	(loves, 0.9799437016320849)	(grandchildren, 0.9487593079781689)	(niece, 0.9186151524791892)	(gift, 0.904124453072548)	(birthday, 0.787330962148161)

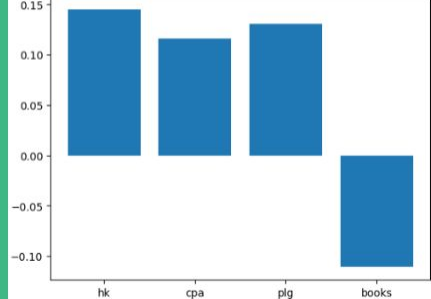
Scores of the word "son"



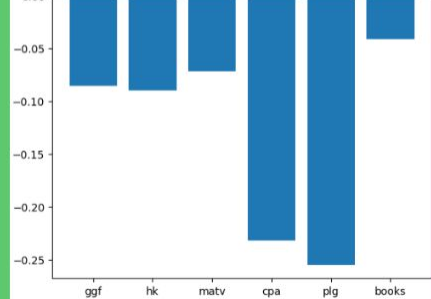
Scores of the word "delivery"



Scores of the word "battery"



Scores of the word "money"





Analyzing Single Reviews

- The program can also analyze just one review
- It works similarly to the testing phase code

Single Review Analysis Code

```
# run this to evaluate only one review

o_r_like_dislike_dict = {}

# consider word threshold for sect[i] to consider
min_word_occurrence = 140
consider_word_threshold = sorted(f_sentiment_dict.items(), key=lambda d: -d[1][1])[0][1][1] + 1
print(consider_word_threshold)

rev_title = str(input("Enter the review title:")).lower()
rev_text = str(input("Enter the review text:")).lower()

# create tokenized instances of review title and text
rev_tokens = [word_tokenize(rev_title), word_tokenize(rev_text)]
sect_num = 0
for sect in rev_tokens:
    # evaluate the sentiment of the target word
    for i in range(len(sect)):
        # evaluate for noun or unknown word
        if is_noun_or_unknown(sect[i]):
            if sect[i] not in stop_words_list:
                if (sect[i] not in f_sentiment_dict) or (f_sentiment_dict[sect[i]][1] < consider_word_threshold):
                    # use the sentiment of surrounding words to weigh the target word
                    for j in range(len(sect)):
                        try:
                            if (sect[j] not in stop_words_list) and (sect[j] in f_sentiment_dict):
                                if sect[i] not in o_r_like_dislike_dict:
                                    if f_sentiment_dict[sect[j]][1] > min_word_occurrence:
                                        o_r_like_dislike_dict[sect[i]] = [f_sentiment_dict[sect[j]][2] / 2**(abs(i - j))], 1]
                                else:
                                    if f_sentiment_dict[sect[j]][1] > min_word_occurrence:
                                        # print(sect[i], like_dislike_dict[sect[i]])
                                        o_r_like_dislike_dict[sect[i]][0] += f_sentiment_dict[sect[j]][2] / (2**(abs(i - j)))
                                        o_r_like_dislike_dict[sect[i]][1] += 1
                                    # print(sect[i], like_dislike_dict[sect[i]])
                        except OverflowError:
                            # print("Overflow encountered while processing " + str(sect[j]) + " for " + str(sect[i]))
                            break
                    if sect_num == 1:
                        for j in range(len(rev_tokens[0])):
                            try:
                                if (rev_tokens[0][j] not in stop_words_list) and (rev_tokens[0][j] in f_sentiment_dict):
                                    if sect[i] not in o_r_like_dislike_dict:
                                        if f_sentiment_dict[rev_tokens[0][j]][1] > min_word_occurrence:
                                            o_r_like_dislike_dict[sect[i]] = [f_sentiment_dict[rev_tokens[0][j]][2], 1]
                                        else:
                                            if f_sentiment_dict[rev_tokens[0][j]][1] > min_word_occurrence:
                                                # print(sect[i], like_dislike_dict[sect[i]])
                                                o_r_like_dislike_dict[sect[i]][0] += f_sentiment_dict[rev_tokens[0][j]][2]
                                                o_r_like_dislike_dict[sect[i]][1] += 1
                                            # print(sect[i], like_dislike_dict[sect[i]])
                            except OverflowError:
                                # print("Overflow encountered while processing " + str(rev_tokens[0][j]) + " for " + str(sect[i]))
                                break
                    sect_num += 1
# append the averages of words into the dictionary
for node in o_r_like_dislike_dict:
    o_r_like_dislike_dict[node].append(o_r_like_dislike_dict[node][0] / o_r_like_dislike_dict[node][1])

# filter the dictionary so there are less irrelevant words
# f_like_dislike_dict = {node : like_dislike_dict[node] for node in like_dislike_dict if like_dislike_dict[node][1] > 350}

# display the results
# print(like_dislike_dict.items())
# The same topic sentiment dictionary is printed twice, but one is sorted from most negative to most positive
# and the other is vice versa
print(sorted(o_r_like_dislike_dict.items(), key = lambda d: d[1][2]))
print(sorted(o_r_like_dislike_dict.items(), key = lambda d: -d[1][2]))
```

```
if f_sentiment_dict[sect[j]][1] > min_word_occurrence:
    # print(sect[i], like_dislike_dict[sect[i]])
    o_r_like_dislike_dict[sect[i]][0] += f_sentiment_dict[sect[j]][2] / (2**(abs(i - j)))
    o_r_like_dislike_dict[sect[i]][1] += 1
    # print(sect[i], like_dislike_dict[sect[i]])
except OverflowError:
    # print("Overflow encountered while processing " + str(sect[j]) + " for " + str(sect[i]))
    break
if sect_num == 1:
    for j in range(len(rev_tokens[0])):
        try:
            if (rev_tokens[0][j] not in stop_words_list) and (rev_tokens[0][j] in f_sentiment_dict):
                if sect[i] not in o_r_like_dislike_dict:
                    if f_sentiment_dict[rev_tokens[0][j]][1] > min_word_occurrence:
                        o_r_like_dislike_dict[sect[i]] = [f_sentiment_dict[rev_tokens[0][j]][2], 1]
                    else:
                        if f_sentiment_dict[rev_tokens[0][j]][1] > min_word_occurrence:
                            # print(sect[i], like_dislike_dict[sect[i]])
                            o_r_like_dislike_dict[sect[i]][0] += f_sentiment_dict[rev_tokens[0][j]][2]
                            o_r_like_dislike_dict[sect[i]][1] += 1
                            # print(sect[i], like_dislike_dict[sect[i]])
                except OverflowError:
                    # print("Overflow encountered while processing " + str(rev_tokens[0][j]) + " for " + str(sect[i]))
                    break
            sect_num += 1
# append the averages of words into the dictionary
for node in o_r_like_dislike_dict:
    o_r_like_dislike_dict[node].append(o_r_like_dislike_dict[node][0] / o_r_like_dislike_dict[node][1])

# filter the dictionary so there are less irrelevant words
# f_like_dislike_dict = {node : like_dislike_dict[node] for node in like_dislike_dict if like_dislike_dict[node][1] > 350}

# display the results
# print(like_dislike_dict.items())
# The same topic sentiment dictionary is printed twice, but one is sorted from most negative to most positive
# and the other is vice versa
print(sorted(o_r_like_dislike_dict.items(), key = lambda d: d[1][2]))
print(sorted(o_r_like_dislike_dict.items(), key = lambda d: -d[1][2]))

102337
Enter the review title:Such an amazing phone!
Enter the review text:This phone works flawlessly! It opens apps on my phone super fast!
[['apps', [12.294357126312578, 10, 1.2294357126312578]], ('phone', [34.87988843097949, 22, 1.5854494741354315])]
[['phone', [34.87988843097949, 22, 1.5854494741354315]], ('apps', [12.294357126312578, 10, 1.2294357126312578])]
```



The Train/Test Data Uses This Format

- Every line of data has three parameters: Rating, Title, and Text
- This is good to know if you want to create your own CSV files for training or testing and you don't want to use my excerpt creation code or want to select reviews from a different corpus

```
1 Rating,Title,Text
2 5.0,Five Stars,CANT WAIT TO GET STARTED....
```




Is There Room for Improvement?

- For a project of this scale, there is only so much room to improve the program
- Other ways to determine if a word is a topic/subtopic
 - Sentence syntax analysis
 - Using a pre-trained model to determine if a word is a subtopic?
- Other ways to determine the sentiment of topics/subtopics
 - Consideration of other factors (such as sentence negation)
 - A pre-trained model could do this task as well



References pg. 1

- <https://huggingface.co/datasets/McAuley-Lab/Amazon-Reviews-2023>
 - The Hugging Face page for McAuley-Lab's 2023 Amazon Reviews dataset
- <https://amazon-reviews-2023.github.io/index.html>
 - The Github website for McAuley-Lab's 2023 Amazon Reviews dataset
- <https://github.com/huggingface/datasets/issues/7693>
 - Advises to downgrade the datasets module version in the event of errors
- <https://huggingface.co/docs/datasets/en/loading>
 - Information on loading Hugging Face and other types of datasets
- <https://www.geeksforgeeks.org/python/reading-and-writing-json-to-a-file-in-python/>
 - Information on handling JSON files in Python
- https://dev.to/scrapfly_dev/jsonl-vs-json-hb0
 - Tells the difference between JSONL and JSON files
- <https://www.geeksforgeeks.org/nlp/removing-stop-words-nltk-python/>
 - Contains an example of using the stopwords and word_tokenize tools from NLTK



References pg. 2

- <https://www.geeksforgeeks.org/python/working-csv-files-python/>
 - Information on handling CSV files in Python
- https://www.w3schools.com/python/python_lambda.asp
 - Explains how the Python lambda keyword works
- <https://www.geeksforgeeks.org/python/python-sort-python-dictionaries-by-key-or-value/>
 - Explains how to sort dictionaries in Python
- <https://www.geeksforgeeks.org/python/python-list-files-in-a-directory/>
 - Tells how to process multiple files in a folder
- https://www.w3schools.com/python/matplotlib_bars.asp
 - A tutorial on creating Matplotlib bar charts
- https://www.w3schools.com/python/matplotlib_labels.asp
 - Tells how to label Matplotlib plots